EJERCICIO: Crear una VPN sencilla con WireGuard

🎯 Objetivo

Aprender a crear una **VPN punto a punto** con **WireGuard**, usando pares de claves públicas y privadas para autenticación.

🧰 Requisitos

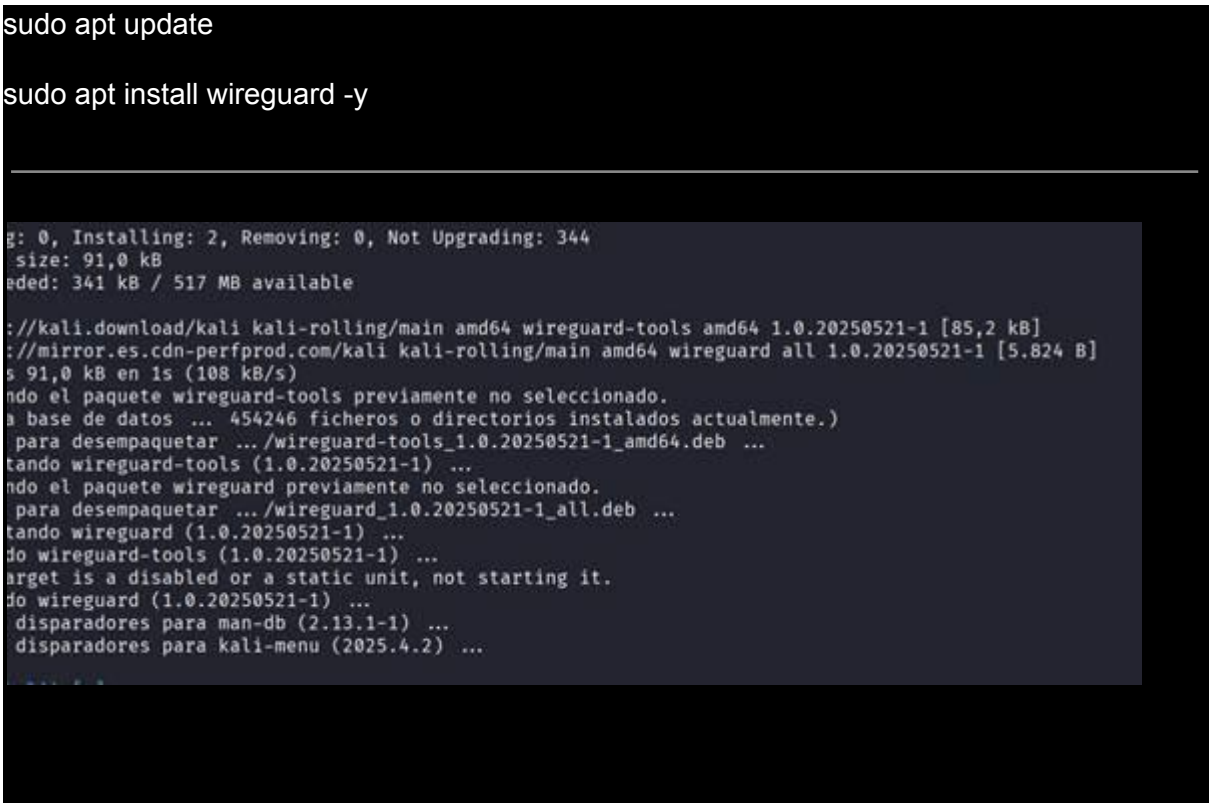
Dos máquinas virtuales Linux (por ejemplo, Kali o Ubuntu):

- una será el **servidor**
- otra el **cliente**

Instala WireGuard en ambas:

```
sudo apt update

sudo apt install wireguard -y
```



- **máquina a:**

```
┌──(kali㉿kali)-[~]
└─$ sudo apt install wireguard -y
The following package was automatically installed and is no longer requi
red:
  python3-proton-vpn-lib
Use 'sudo apt autoremove' to remove it.

Installing:
  wireguard

Summary:
  Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 347
  Download size: 5,824 B
  Space needed: 15.4 kB / 16.1 GB available

Get:1 http://kali.download/kali kali-rolling/main amd64 wireguard all 1.
0.20250521-1 [5,824 B]
Fetched 5,824 B in 0s (15.1 kB/s)
Selecting previously unselected package wireguard.
(Reading database ... 505482 files and directories currently installed.)
Preparing to unpack .../wireguard_1.0.20250521-1_all.deb ...
Unpacking wireguard (1.0.20250521-1) ...
Setting up wireguard (1.0.20250521-1) ...
```

*** máquina b:**

```
┌──(kali㉿kali)-[~]
└─$ sudo apt install wireguard -y
Los paquetes indicados a continuación se instalaron de forma automática y ya no
on necesarios.
  amass-common               libx264-164
  libadwaita-1-0             libxml2
  libbluray2                 libyelp-1-0
  libbson-1.0-0t64           python3-bluepy
  libjavascriptcoregtk-6.0-1 python3-click-plugins
  libjs-jquery-ui            python3-gpg
  libjs-underscore           python3-kismetcapturebtgeiger
  libmongoc-1.0-0t64         python3-kismetcapturefreaklabszigbee
  libmongocrypt0             python3-kismetcapturertl433
  libplacebo349              python3-kismetcapturertladsb
  libportmidi0               python3-kismetcapturertlamr
  librav1e0.7                python3-protobuf
  libtheoradec1              python3-zombie-imp
  libtheoraenc1              samba-ad-dc
  libudfread0                samba-ad-provision
  libwebkitgtk-6.0-4         samba-dsdb-modules
Utilice «sudo apt autoremove» para eliminarlos.

Upgrading:
  ruby-snmp

Installing:
  wireguard
```

🔹 **1. Crear las claves**

En cada máquina (servidor y cliente):

```
wg genkey | tee privatekey | wg pubkey > publickey
```

Esto genera:

- privatekey → clave privada
- publickey → clave pública
- si queremos verlas podemos poner # Si estás en la misma carpeta donde las generaste:

```
cat privatekey
cat publickey
```

📍 Guarda esas claves, las necesitarás para configurar los peers.

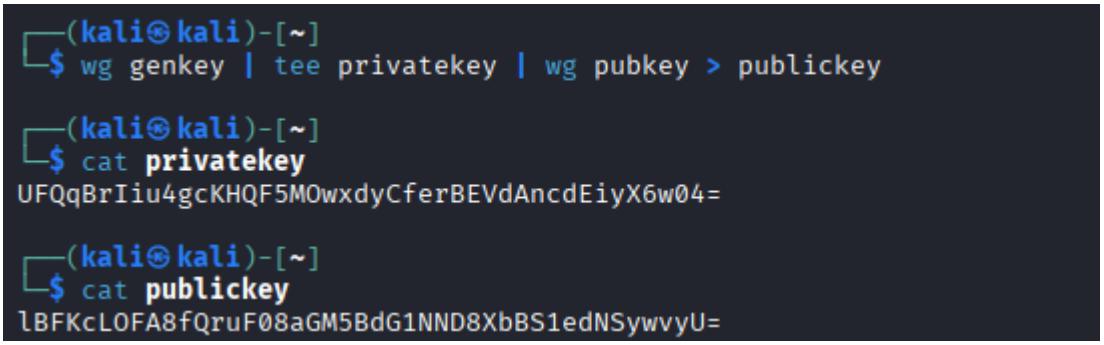
- **máquina a:**

cat privatekey

UFQqBrIiu4gcKHQF5MOwxdyCferBEVdAncdEiyX6w04=

cat publickey

lBFKcLOFA8fQruF08aGM5BdG1NND8XbBS1edNSywvyU=



- **máquina b:**

cat privatekey

YBm1JKbMpmQYIUIMOtiQmsp8Eq/QrQyV2yL8gitmlIA=

cat publickey

Ca03csntwNAkRQZhxGKo64A7ThiJn388bPNYp4pemm8=

```
┌──(kali㉿kali)-[~]
└─$ wg genkey | tee privatekey | wg pubkey > publickey

┌──(kali㉿kali)-[~]
└─$ cat privatekey
YBm1JKbMpmQYIUIMOtiQmsp8Eq/QrQyV2yL8gitmllA=

┌──(kali㉿kali)-[~]
└─$ cat publickey
Ca03csntwNAkRQZhxGKo64A7ThiJn388bPNYp4pemm8=
```

🔹 **2. Configurar el servidor**

Edita el archivo /etc/wireguard/wg0.conf:

```
sudo nano /etc/wireguard/wg0.conf
```

Pega esto (ajusta la IP de red y el puerto):

```
[Interface]

Address = 10.10.0.1/24

ListenPort = 51820

PrivateKey = (aquí la clave privada del servidor)



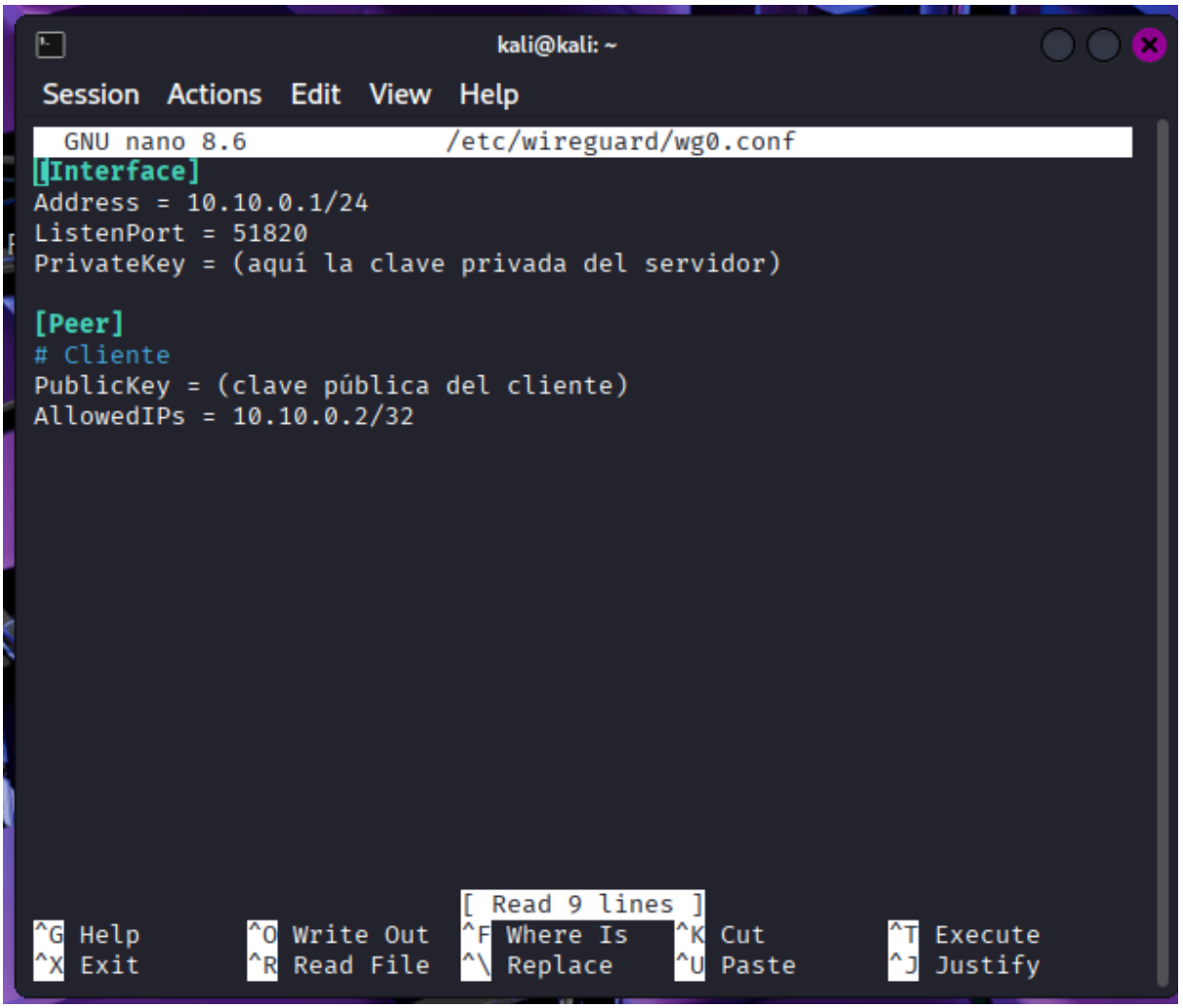
[Peer]

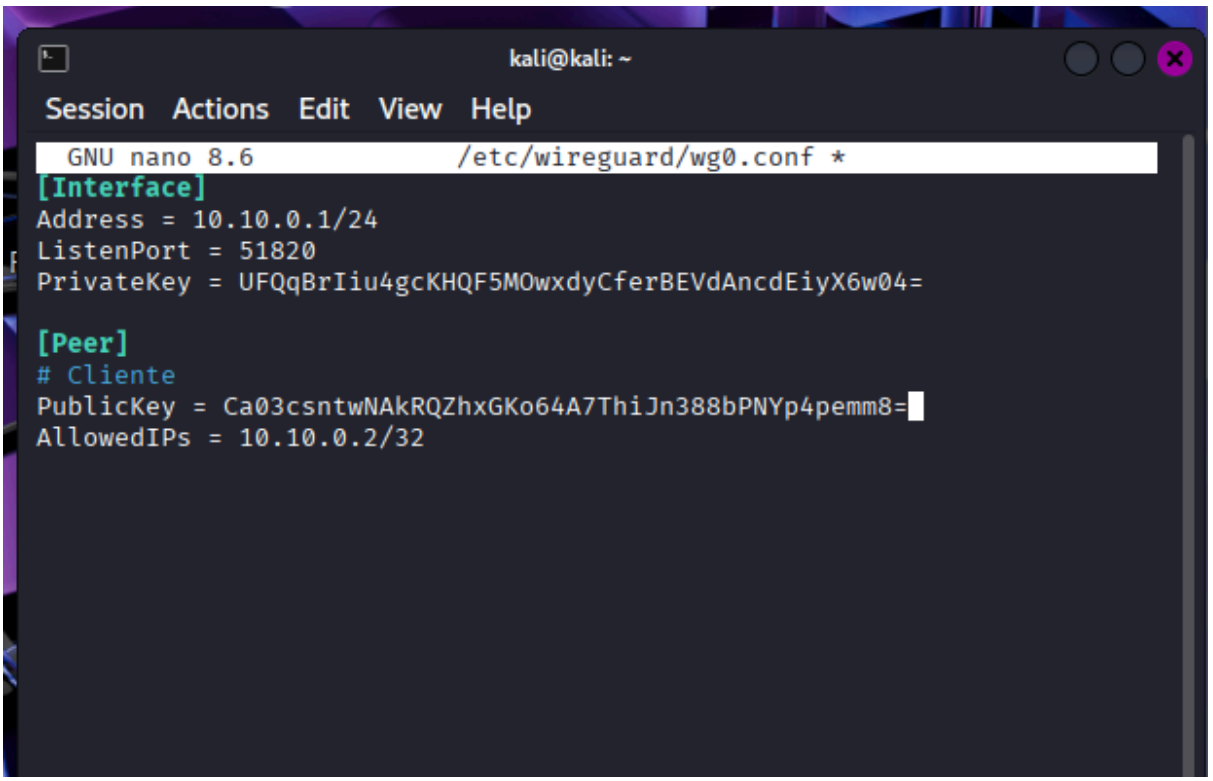
# Cliente

PublicKey = (clave pública del cliente)
```

```
AllowedIPs = 10.10.0.2/32
```

Guarda y cierra (**Ctrl + O**, Enter, **Ctrl + X**).





Hay que mirar la clave privada del servidor:

La clave publica del cliente:

- **3. Configurar el cliente**

En el cliente, edita:

```
sudo nano /etc/wireguard/wg0.conf
```

Pega esto:

```
[Interface]

Address = 10.10.0.2/24

PrivateKey = (clave privada del cliente)
```

```
[Peer]

# Servidor

PublicKey = (clave pública del servidor)

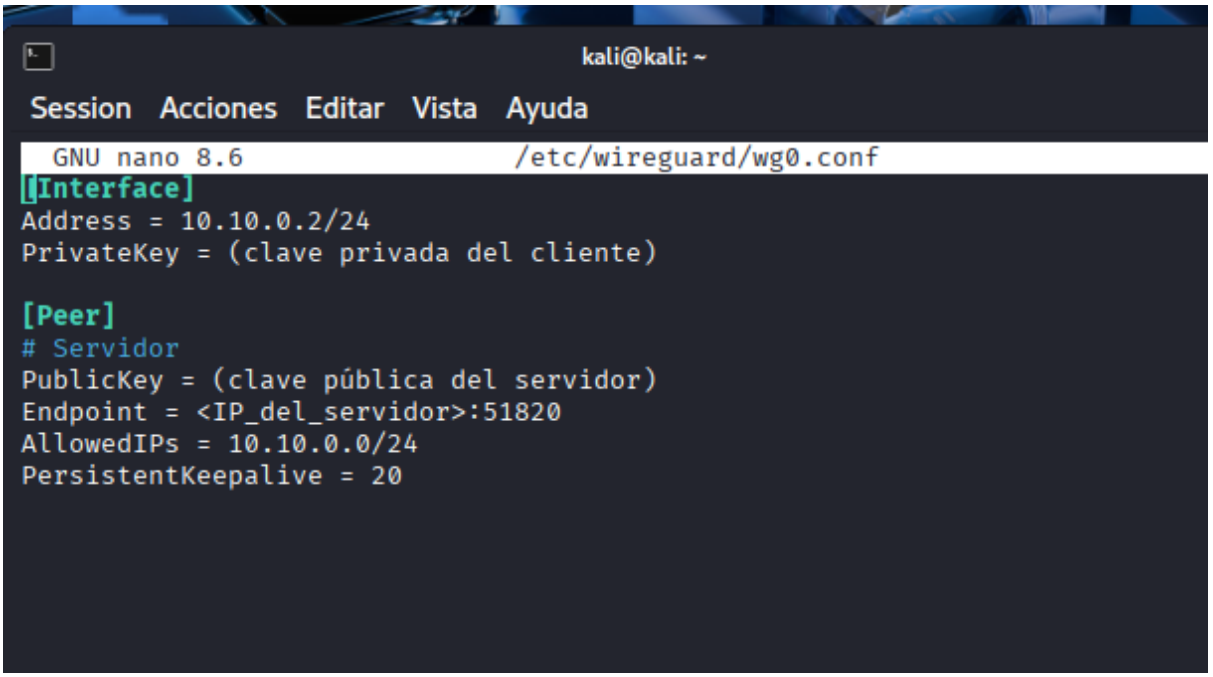
Endpoint = <IP_del_servidor>:51820
```

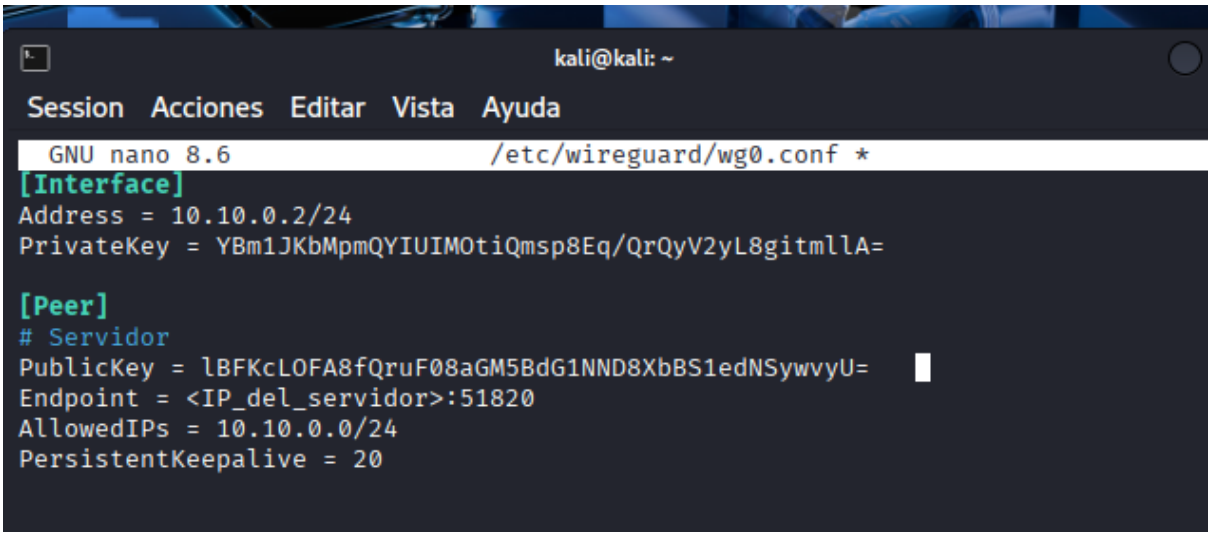
AllowedIPs = 10.10.0.0/24

PersistentKeepalive = 20

Mirar la clave privada del cliente

La clave publica del servidor





- **4. Activar la VPN**

En ambas máquinas:

```
sudo wg-quick up wg0
```

✅ Si todo está bien, verás algo como:

```
[#] ip link add wg0 type wireguard

[#] wg setconf wg0 /dev/fd/63

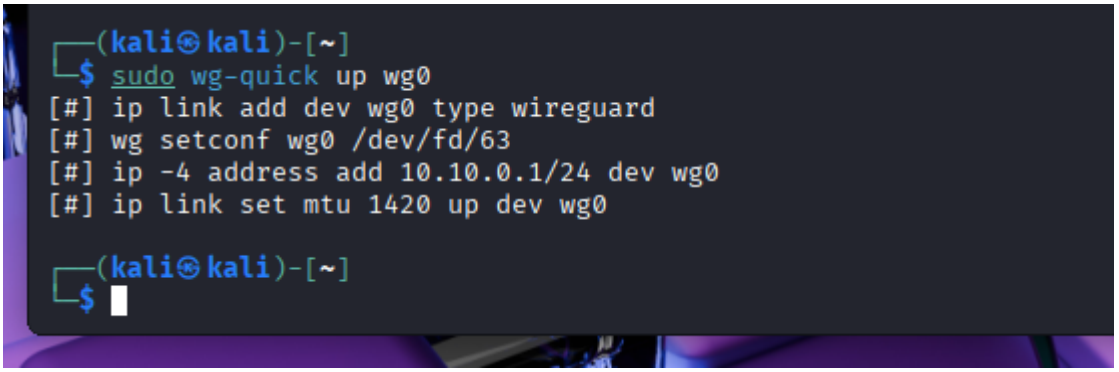
[#] ip -4 address add 10.10.0.1/24 dev wg0

[#] ip link set mtu 1420 up dev wg0

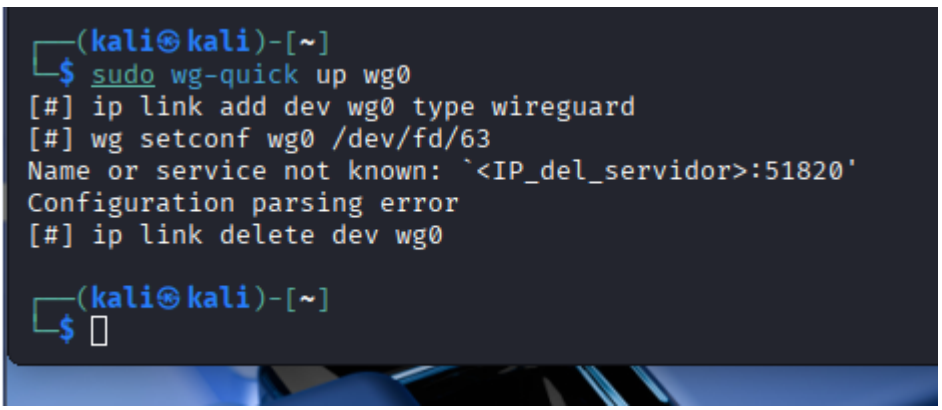
[#] ip -4 route add 10.10.0.2/32 dev wg0
```

---

máquina a:



máquina b:

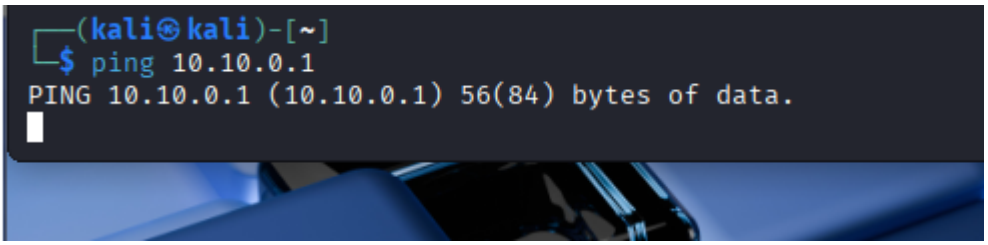


- **5. Comprobar la conexión**

En el cliente:

```
ping 10.10.0.1
```

```
osboxes@osboxes:~$ ping 10.10.0.1
PING 10.10.0.1 (10.10.0.1) 56(84) bytes of data.
64 bytes from 10.10.0.1: icmp_seq=1 ttl=64 time=1.09 ms
64 bytes from 10.10.0.1: icmp_seq=2 ttl=64 time=0.035 ms
64 bytes from 10.10.0.1: icmp_seq=3 ttl=64 time=0.037 ms
64 bytes from 10.10.0.1: icmp_seq=4 ttl=64 time=0.040 ms
64 bytes from 10.10.0.1: icmp_seq=5 ttl=64 time=0.039 ms
64 bytes from 10.10.0.1: icmp_seq=6 ttl=64 time=0.053 ms
^X64 bytes from 10.10.0.1: icmp_seq=7 ttl=64 time=0.038 ms
64 bytes from 10.10.0.1: icmp_seq=8 ttl=64 time=0.037 ms
^C
--- 10.10.0.1 ping statistics ---
8 packets transmitted, 8 received, 0% packet loss, time 9080ms
rtt min/avg/max/mdev = 0.035/0.171/1.092/0.348 ms
```



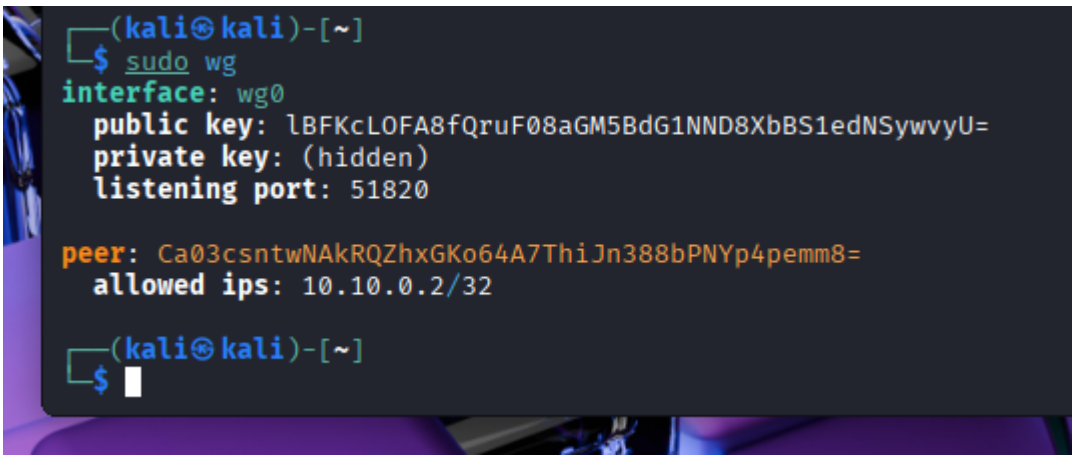
Si responde, la VPN está funcionando 🎉

Puedes ver el estado:

```
sudo wg
```

```
sboxes@osboxes:~$ sudo wg
nterface: wg0
 public key: h4VpThYeR+A5nP9gUF2oxz0eeXNeadvZDRiFtSN57ww=
 private key: (hidden)
 listening port: 51820

eer: U5gIndpSa8QkyhDXfjDdtlFdH7EVxUo2APxhTpNYigI=
 allowed ips: 10.10.0.2/32
```


┌──(kali㉿kali)-[~]
└─$ sudo wg
interface: wg0
public key: lBFKcLOFA8fQruF08aGM5BdG1NND8XbBS1edNSywvyU=
private key: (hidden)
listening port: 51820
peer: Ca03csntwNAkRQZhxGKo64A7ThiJn388bPNYp4pemm8=
allowed ips: 10.10.0.2/32
┌──(kali㉿kali)-[~]
└─$